Métodos de arrays y funciones flecha

map · filter · find · some · every

PUNTO DE PARTIDA

El resultado que necesitamos

Un array con elementos transformados

Un array con los elementos seleccionados

Un elemento que cumpla una condición

Un booleano que responda a una pregunta

El catálogo de la sesión anterior

```
const videojuegos = [
  { id: 1, titulo: "Hollow Knight", genero: "Metroidvania",
    precio: 15, disponible: true },
  { id: 2, titulo: "Celeste", genero: "Plataformas",
    precio: 20, disponible: true },
  { id: 3, titulo: "Hades", genero: "Roguelike",
    precio: 25, disponible: false },
{ id: 4, titulo: "Stardew Valley", genero: "Simulación",
    precio: 14, disponible: true },
  { id: 5, titulo: "Dead Cells", genero: "Roguelike",
    precio: 24, disponible: false }
];
```

Una colección de objetos con la misma estructura.

El catálogo, continuación

```
  { id: 4, titulo: "Stardew Valley", genero: "Simulación",
    precio: 14, disponible: true },
  { id: 5, titulo: "Dead Cells", genero: "Roguelike",
    precio: 24, disponible: false }
];
```

Reto: obtener los títulos y, por separado, los disponibles.

La transformación que ya conocemos

```
const titulos = [];
for (const videojuego of videojuegos) {
    titulos.push(videojuego.titulo);
}
```

Cada objeto aporta un título al nuevo array.

La selección que ya conocemos

```
const disponibles = [];
for (const videojuego of videojuegos) {
    if (videojuego.disponible) {
        disponibles.push(videojuego);
    }
}
```

Solo entran los objetos que cumplen la condición.

Tres formas de escribir la función

```
function obtenerTitulo(videojuego) {
    return videojuego.titulo;
}

const obtenerTitulo = (videojuego) => {
    return videojuego.titulo;
};

const obtenerTitulo = videojuego => videojuego.titulo;
```

Son alternativas: ejecutar una sola declaración cada vez.

Una función como argumento

```
function mostrarResultado(elemento, operacion) {
    console.log(operacion(elemento));
}

mostrarResultado(videojuegos[0], obtenerTitulo);
```

obtenerTitulo representa la función. No la ejecuta todavía.

El return que falta

```
const obtenerPrecio = videojuego => {
    videojuego.precio;
};

console.log(obtenerPrecio(videojuegos[0]));
```

¿Qué devuelve esta función?

map transforma cada elemento

```
const titulos = videojuegos.map(
    videojuego => videojuego.titulo
);

console.log(titulos);
```

Resultado: un array nuevo de títulos.
* La función callback del map no se ejecuta sobre los huecos, pero el array que devuelve, si lleva el hueco.

El callback decide qué se guarda

```
const preciosConRecargo = videojuegos.map(
    videojuego => videojuego.precio + 5
);

console.log(preciosConRecargo);
// [20, 25, 30, 19, 29]
```

El resultado contiene números. Los precios originales no cambian.

Mensajes a partir de los objetos

```
const mensajes = videojuegos.map(
    videojuego =>
        `${videojuego.titulo} cuesta ${videojuego.precio} €`
);
```

Ejemplo: "Celeste cuesta 20 €"

filter selecciona elementos

```
const disponibles = videojuegos.filter(
    videojuego => videojuego.disponible
);
```

Devuelve un array: Hollow Knight, Celeste y Stardew Valley.

Una condición compuesta

```
const disponiblesBaratos = videojuegos.filter(
    videojuego =>
        videojuego.disponible && videojuego.precio < 20
);
```

Cumplen las dos condiciones: Hollow Knight y Stardew Valley.

find obtiene la primera coincidencia

```
const encontrado = videojuegos.find(
    videojuego => videojuego.id === 4
);

console.log(encontrado.titulo);
// Stardew Valley
```

Resultado: un objeto. Si no hay coincidencias: undefined.

filter y find devuelven cosas distintas

```
const lista = videojuegos.filter(v => v.id === 4);
// [ { id: 4, titulo: "Stardew Valley", ... } ]

const elemento = videojuegos.find(v => v.id === 4);
// { id: 4, titulo: "Stardew Valley", ... }
```

filter devuelve un array. find devuelve un elemento o undefined.

Una búsqueda sin coincidencias

```
const encontrado = videojuegos.find(v => v.id === 99);

if (encontrado !== undefined) {
    console.log(encontrado.titulo);
} else {
    console.log("Videojuego no encontrado");
}
```

La comprobación evita acceder a una propiedad de undefined.

PRÁCTICA BREVE

Tres consultas para elegir el método

Todos los videojuegos de género "Roguelike"

El primer videojuego que cueste más de 20 €

Los disponibles cuyo precio sea como máximo 20 €

some comprueba si alguno cumple

```
const hayNoDisponibles = videojuegos.some(
    videojuego => !videojuego.disponible
);

console.log(hayNoDisponibles);
// true
```

Devuelve un booleano y termina al encontrar una coincidencia.

every comprueba si todos cumplen

```
const todosDisponibles = videojuegos.every(
    videojuego => videojuego.disponible
);

console.log(todosDisponibles);
// false
```

Devuelve un booleano y termina cuando uno no cumple.

Tres predicciones antes de ejecutar

```
videojuegos.some(v => v.precio < 15);

videojuegos.every(v => v.precio > 0);

videojuegos.some(v => v.genero === "Deportes");
```

¿Qué devuelve cada expresión?

El caso del array vacío

```
[].some(v => v.disponible);  // false
[].every(v => v.disponible); // true

const todosYHayElementos =
    videojuegos.length > 0 &&
    videojuegos.every(v => v.disponible);
```

Si también debe haber elementos, comprobar length.

Práctica 1–5

1. Obtener un array con todos los títulos
2. Crear mensajes como "Celeste - 20 €"
3. Obtener los videojuegos de menos de 20 €
4. Obtener los disponibles de como máximo 20 €
5. Buscar el id "4", recibido como texto

Práctica 6–10

6. Buscar el id 99 y gestionar su ausencia

7. Comprobar si hay algún Roguelike no disponible

8. Comprobar si todos los precios están entre 0 y 25 € sin incluir 0 e incluyendo 25
9. Obtener los títulos de los videojuegos disponibles

10. Crear seleccionar(lista, criterio) con filter

Corrección: el tipo del identificador

```
const idBuscado = "4";
const encontrado = videojuegos.find(
    videojuego => videojuego.id === Number(idBuscado)
);
```

"4" es string. Los identificadores del catálogo son number.

Selección seguida de transformación

```
const disponibles = videojuegos.filter(
    videojuego => videojuego.disponible
);

const titulosDisponibles = disponibles.map(
    videojuego => videojuego.titulo
);
```

Primero obtenemos objetos. Después obtenemos sus títulos.

Las mismas operaciones encadenadas

```
const titulosDisponibles = videojuegos
    .filter(videojuego => videojuego.disponible)
    .map(videojuego => videojuego.titulo);
```

map trabaja sobre el array que devuelve filter.

Una función que recibe el criterio

```
function seleccionar(lista, criterio) {
    return lista.filter(criterio);
}

const baratos = seleccionar(
    videojuegos, videojuego => videojuego.precio < 20
);
```

criterio es una función que filter aplicará a los elementos.

Un array nuevo puede compartir objetos

```
const disponibles = videojuegos.filter(v => v.disponible);

disponibles[0].precio = 99;

console.log(videojuegos[0].precio);
// 99
```

Los dos arrays contienen una referencia al mismo objeto.

La elección depende del resultado

map: un array con los resultados de la transformación

filter: un array con los elementos seleccionados

find: el primer elemento encontrado o undefined

some: true si al menos uno cumple

every: true si todos cumplen